CSC 1102: Structured and Object-Oriented Programming
Practical Exam Project

Student Marks Management System

Group 24

Group leader: Lubega Michael		
Name	**Student No.**	**Registration No.**
Sebalemba Dennis	2600726046	26/U/26046/PS
Lubega Michael	2600703548	26/U/03548/PS
Muyonjo Solomon	2600703581	26/U/03581/PS
Rob Wilbert	2600726537	26/X/26537/PS
Ssenfuma Emmanuel	2600703644	26/U/03644/PS

GitHub link: https://github.com/c-eo/oop-practical

1. Description of the application

The Student Marks Management System is a desktop application written in Java with a Swing graphical user interface. It lets a lecturer or administrator record students and their marks in six subjects, and it automatically calculates each student's average mark and letter grade. Students are stored in an array of objects, so no database is needed. The application is started by running the `StudentGUI` class.

The six subjects are Object-Oriented Programming, Mathematics, Digital Innovation, Computer Organisation and Architecture, Computer Literacy, and Communication Skills. Each mark is a whole number from 0 to 100.

2. Main features

- **Add Student:** enter a name, a 10-digit student number and six marks. The record is stored in the array and shown in the table.
- **Input validation:** names, student numbers and marks are checked against clear rules and an error message is shown for invalid input.
- **Automatic average and grade:** the average of the six marks and its grade (A+ to F) are calculated for every student, and each subject mark is also shown with its grade.
- **Live Search:** the table is filtered as the user types in the search box; the Clear button resets it.
- **View Report:** shows a full report of the selected student with every mark, its grade and the average.
- **Update Student:** opens a dialog to edit the selected student's name and marks.
- **Delete Selected:** removes the selected student and closes the gap in the array.
- **Sort by Average:** orders the students from the highest to the lowest average; a rank is shown in the table.
- **Clear Form / Exit:** reset the entry form, or close the application after confirmation.

3. Classes and their purpose

Class	Purpose
`Student`	Base class. Stores the student's name and student number as private attributes, with a constructor, getters, and a validated setter for the name. There is no setter for the student number because it never changes.
`Marks`	Extends `Student`. Stores the six subject marks and the average. Calculates the average, provides getters, a validated `setAllMarks` method, and the static `gradeCalc` method that converts a mark to a grade.
`Details`	Extends `Marks`. Represents one complete student record. Adds `getReport()` for the full report and overrides `toString()` for a one-line summary.
`Validator`	Utility class with static methods holding every validation rule in one place. Each check returns an empty string when the input is valid or an error message otherwise.

`StudentGUI`	Main class. Extends `JFrame` and implements `ActionListener`. Builds the Swing interface, holds the `Details[]` array and handles every button action.

4. How the classes interact

Inheritance. The data classes form a chain, `Student` → `Marks` → `Details`. Each child calls its parent's constructor with `super(...)`, so creating one `Details` object sets up the name, the number, the marks and the average together.

```
Student  (name, number)
   ^
Marks    (six marks, average, grades)
   ^
Details  (full record: report, toString)

StudentGUI --creates/stores--> Details[] students
StudentGUI, Student, Marks --call--> Validator
```

The GUI and the objects. `StudentGUI` keeps the records in the array `Details[] students` (up to 50) with a counter `studentCount`. When the user clicks Add Student, the GUI validates the form, creates a new `Details` object and stores it in the array. Update calls the setters, Delete shifts the array elements left, and Sort reorders the array by average using a selection sort. The table is rebuilt from the array by `refreshTable()` after every change.

Validation. `StudentGUI` calls `Validator` on the raw text typed by the user before any object is created. `Student` and `Marks` also use `Validator` inside their setters, so a record can never be changed to an invalid value.

5. Programming concepts demonstrated

Concept	Where it is used
Classes and objects	`Details` objects are created for every student.
Constructors	Parameterised constructors in `Student`, `Marks` and `Details`.
Access modifiers and encapsulation	Private attributes accessed through public getters and setters.
Inheritance and overriding	`Student` → `Marks` → `Details`; `toString()` is overridden.
Arrays	`Details[] students` stores and manages all records.
Conditionals	if/else chains in `gradeCalc` and in the validation methods.
Loops	for loops for sorting, searching, deleting and checking input characters.
Swing GUI and events	`JFrame`, `JTable`, dialogs, buttons and `ActionListener`.